EX NO. : 1 **Implement Data similarity measures using Python.**
DATE:

AIM:

To implement data similarity measures using python.

ALGORITHM:

1. Import the necessary libraries.
2. Define the function.
3. Get the data and implement in the formula.
4. Print the output of the data.

PROGRAM:

Numeric Data:-

1. **Euclidean Distance:**

```
import numpy as np
def euclidean_distance(x, y):
    return np.sqrt(np.sum((x - y) ** 2))
# Example usage:
data1 = np.array([1, 2, 3])
data2 = np.array([4, 5, 6])
distance = euclidean_distance(data1, data2)
    print(f"Euclidean Distance: {distance}")
```

2. Cosine Similarity:

```
from sklearn.metrics.pairwise import cosine_similarity
data1 = np.array([1, 2, 3])
data2 = np.array([4, 5, 6])
```

```
# Reshape data to be 2D arrays
data1 = data1.reshape(1, -1)
data2 = data2.reshape(1, -1)

cosine_sim = cosine_similarity(data1, data2)
    print(f"Cosine Similarity: {cosine_sim[0][0]}")
```

Text Data:-

3. Jaccard Similarity:

```
def jaccard_similarity(set1, set2):
        intersection = len(set1.intersection(set2))
        union = len(set1.union(set2))
        return intersection / union

# Example usage:
text1 = set("hello world")
text2 = set("world hello")

similarity = jaccard_similarity(text1, text2)
print(f"Jaccard Similarity: {similarity}")
```

4. Levenshtein Distance(Edit Distance):

```
def levenshtein_distance(str1, str2):
   if len(str1) > len(str2):
      str1, str2 = str2, str1

   distances = range(len(str1) + 1)
   for index2, char2 in enumerate(str2):
      new_distances = [index2 + 1]
      for index1, char1 in enumerate(str1):
         if char1 == char2:
            new_distances.append(distances[index1])
                  else:
             new_distances.append(1 + min((distances[index1], distances[index1 + 1],
           new_distances[-1])))
      distances = new_distances
   return distances[-1]
```

```
# Example usage:
word1 = "kitten"
word2 = "sitting"
distance = levenshtein_distance(word1, word2)
    print(f"Levenshtein Distance: {distance}")
```

OUTPUT:

1. Euclidean Distance: 5.196152422706632
2. Cosine Similarity: 0.9746318461970762
3. Jaccard Similarity: 1.0
4. Levenshtein Distance: 3

RESULT:

Thus the implementation of data similarity measures using python was executed successfully.

EX.NO.:2 **Implement dimension reduction techniques for recommender systems.**
DATE:

AIM:

To implement dimension reduction techniques for recommender systems.

ALGORITHM:

Singular Value Decomposition:-

1. Import the necessary libraries
2. Generate sample data matrix
3. Split the data into training and testing datasets
4. Perform Singular value decomposition
5. Reconstruct the Original data matrix from the reduced form
6. Calculate Mean Squared Error on the test set

Matrix Factorization:-

1. Import the necessary libraries
2. Load the dataset
3. Use the dataset to form a dataframe format
4. Use the SVD algorithm
5. Perform cross-validation
6. Print the CV

PROGRAM:

Singular Value Decomposition:-

```
import numpy as np
from sklearn.decomposition import TruncatedSVD
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split
from scipy.sparse import lil_matrix
# Generate sample user-item matrix (replace this with your data)
num_users = 100
num_items = 50
ratings = np.random.randint(1, 6, size=(num_users, num_items))
# Split data into training and testing sets
train_data, test_data = train_test_split(ratings, test_size=0.2, random_state=42)
# Perform Singular Value Decomposition
n_components = 10  # Set the number of components (latent factors)
svd = TruncatedSVD(n_components=n_components)
reduced_train_data = svd.fit_transform(train_data)
# Reconstruct the original matrix from the reduced form
reconstructed_train_data = np.dot(reduced_train_data, svd.components_)
# Calculate Mean Squared Error on the test set
mse = mean_squared_error(train_data, reconstructed_train_data)
print(f"Mean Squared Error: {mse}")
```

Matrix Factorization:-

```
from surprise import SVD
from surprise import Dataset
from surprise.model_selection import cross_validate
```

 # Load your dataset (replace this with your data)

 # Use the load_from_df method if your data is in DataFrame format

data = Dataset.load_builtin('ml-100k')

 # Use the SVD algorithm

algo = SVD()

 # Perform cross-validation

CV=cross_validate(algo, data, measures=['RMSE'], cv=5, verbose=True)

print(CV)

OUTPUT:

Singular Value Decomposition:-

Mean Squared Error: 1.0896028841627086

Matrix Factorization:-

	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Mean	Std
RMSE (test_set)	0.9306	0.9388	0.9348	0.9399	0.9365	0.9361	0.0033
Fit time	2.04	1.76	1.84	2.43	1.55	1.92	0.30
Test time	0.18	0.22	0.23	0.37	0.14	0.23	0.08

{'test_rmse': array([0.93062173, 0.93880913, 0.93483342, 0.93991998, 0.93651167]),

'fit_time': (2.036858320236206, 1.7616021633148193, 1.8352279663085938, 2.430487871170044, 1.5543479919433594),

'test_time': (0.1761641502380371, 0.22463703155517578, 0.22916793823242188, 0.3696250915527344, 0.14364123344421387)}

RESULT:

Thus, the implementation of dimension reduction techniques for recommender systems was executed successfully.

EX NO.:3 **Implement user-profile learning**
DATE:

AIM:

To implement user-profile learning.

ALGORITHM:

1. Import the necessary libraries
2. Get the sample user-data
3. Convert the user-data into matrix
4. Calculate the cosine similarity between users
5. Perform functions to get the personalized recommendations for a given user
6. Find the highest similarity
7. Aggregate items liked by similar users
8. Remove the items already liked by user
9. Print the recommendations

PROGRAM:

```
import numpy as np
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.metrics.pairwise import cosine_similarity
# Sample user-item interaction data (replace this with your data)
user_item_data =
{
    'user1': 'item1 item2 item4',
    'user2': 'item2 item3 item5',
    'user3': 'item1 item3 item4',
    'user4': 'item2 item4 item5',
}
```

```
# Convert the user-item interactions into a matrix representation (Bag-of-Words)
vectorizer = CountVectorizer(binary=True)
user_item_matrix = vectorizer.fit_transform(user_item_data.values())
# Calculate cosine similarity between users
user_similarity_matrix = cosine_similarity(user_item_matrix, user_item_matrix)
# Function to get personalized recommendations for a given user
def get_recommendations(user, user_similarity_matrix, user_item_data, n_recommendations=2):
    user_index = list(user_item_data.keys()).index(user)
    similarities = user_similarity_matrix[user_index]
        # Find the indices of users with highest similarity (excluding the user itself)
    similar_users_indices = np.argsort(similarities)[::-1][1:n_recommendations+1]
    # Aggregate items liked by similar users
    recommended_items = set()
    for index in similar_users_indices:
        items = user_item_data[list(user_item_data.keys())[index]].split()
        recommended_items.update(items)
    # Remove items already liked by the user
    user_items = user_item_data[user].split()
    recommended_items -= set(user_items)
    return recommended_items
# Example: Get recommendations for 'user1'
user_to_recommend = 'user1'
recommendations = get_recommendations(user_to_recommend, user_similarity_matrix, user_item_data)
print(f"Recommendations for {user_to_recommend}: {recommendations}")
```

OUTPUT:

Recommendations for user1: {'item5', 'item3'}

RESULT:

Thus the implementation of user-profile learning. was executed successfully.

EX. NO.:4 **Implement content-based recommendation systems.**
DATE:

AIM:

To implement content-based recommendation systems.

ALGORITHM:

1. Import the necessary libraries

2. Get the sample data

3. Get the sample user data

4. Convert text data into TF-IDF features

5. Calculate the cosine similarity between items and user preference

6. Perform the function to get content-based recommendations for a user (weighted sum of items, indices of items, return recommended items)

7. Print the content-based recommendations

PROGRAM:

```
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import linear_kernel
# Sample item data (replace this with your data)
items = pd.DataFrame({
    'item_id': [1, 2, 3, 4],
    'title': ['Action Movie', 'Comedy Movie', 'Drama Movie', 'Sci-Fi Movie'],
    'genre': ['Action', 'Comedy', 'Drama', 'Sci-Fi'],
    'description': ['Explosions and car chases.', 'Laughs and humor all the way.', 'Intense emotional scenes.', 'Futuristic technology and space adventures.']})
# Sample user preferences (replace this with your data)
user_preferences = {
        'Action': 5,
        'Comedy': 4,
        'Drama': 2,
        'Sci-Fi': 3
        }
```

```
# Convert text data (title, genre, description) to TF-IDF features
tfidf_vectorizer = TfidfVectorizer(stop_words='english')
item_features = tfidf_vectorizer.fit_transform(items['title'] + ' ' + items['genre'] + ' ' + items['description'])
# Calculate cosine similarity between items and user preferences
cosine_similarities = linear_kernel(item_features, tfidf_vectorizer.transform([f"{pref}" for pref in
user_preferences.keys()]))
# Function to get content-based recommendations for a user
def get_content_based_recommendations(user_preferences, items, item_features, cosine_similarities,
n_recommendations=2):
    # Weighted sum of item similarities based on user preferences
    weighted_similarities = np.dot(cosine_similarities.T, list(user_preferences.values()))
      # Get indices of items with highest weighted similarity
    recommended_item_indices = weighted_similarities.argsort()[::-1][:n_recommendations]
    # Return recommended items
    recommendations = items.iloc[recommended_item_indices]
    return recommendations
# Example: Get content-based recommendations for the user
content_based_recommendations = get_content_based_recommendations(user_preferences, items,
item_features, cosine_similarities)
print("Content-Based Recommendations:")
print(content_based_recommendations[['item_id', 'title', 'genre', 'description']])
```

OUTPUT:

Content-Based Recommendations:

	item_id	title	genre	description
0	1	Action Movie	Action	Explosions and car chases.
1	2	Comedy Movie	Comedy	Laughs and humor all the way.

RESULT:

Thus the implementation of content-based recommendation systems was executed successfully.

EX.NO.:5 **Implement collaborative filter techniques.**
DATE:

Aim:

To implement collaborative filter techniques.

ALGORITHM:

User-based collaborative filtering:-

1. Import the necessary libraries
2. Sample user-item rating data
3. Create a user-item matrix
4. Calculate cosine similarity between users
5. Perform to get collaborative filtering recommendations for a user (Users with highest similarity, items that the similar users liked and the current user hasn't)
6. Print User-based recommendations

Item-based collaborative filtering: -

1. Calculate cosine similarity between items
2. Perform the function to get item-based collaborative filtering recommendations (Find items that user has not rated, calculate the average weighted average of item ratings, sort items by predicted score, get the top recommendations)
3. Print Item-based recommendations

PROGRAM:

User-based collaborative filtering:-

```
import pandas as pd

from sklearn.metrics.pairwise import cosine_similarity
```

```
# Sample user-item rating data (replace this with your data)
ratings_data = pd.DataFrame({
    'user_id': [1, 1, 2, 2, 3, 3, 4, 4],
    'item_id': [1, 2, 2, 3, 3, 4, 1, 4],
    'rating': [5, 4, 5, 3, 4, 2, 3, 5]
})
# Create a user-item matrix
user_item_matrix = ratings_data.pivot_table(index='user_id', columns='item_id', values='rating', fill_value=0)
# Calculate cosine similarity between users
user_similarity_matrix = cosine_similarity(user_item_matrix)
# Function to get collaborative filtering recommendations for a user
def get_user_based_recommendations(user_id, user_item_matrix, user_similarity_matrix, n_recommendations=2):
    user_index = user_id - 1  # Adjust index to start from 0
    similarities = user_similarity_matrix[user_index]
    # Find the indices of users with highest similarity (excluding the user itself)
    similar_users_indices = similarities.argsort()[::-1][1:n_recommendations+1]
# Get items that the similar users liked and the current user hasn't
    recommended_items = []
    for index in similar_users_indices:
        liked_items = user_item_matrix.iloc[index][user_item_matrix.iloc[index] > 0].index
        user_items = user_item_matrix.iloc[user_index][user_item_matrix.iloc[user_index] > 0].index
        new_items = set(liked_items) - set(user_items)
        recommended_items.extend(new_items)
    return recommended_items[:n_recommendations]
```

```
# Example: Get user-based collaborative filtering recommendations for user 1

user_id_to_recommend = 1

user_based_recommendations = get_user_based_recommendations(user_id_to_recommend, user_item_matrix, user_similarity_matrix)

print(f"User-Based Collaborative Filtering Recommendations for User {user_id_to_recommend}: {user_based_recommendations}")
```

Item-based collaborative filtering:-

```
# Calculate cosine similarity between items

item_similarity_matrix = cosine_similarity(user_item_matrix.T)

# Function to get item-based collaborative filtering recommendations for a user

def get_item_based_recommendations(user_id, user_item_matrix, item_similarity_matrix, n_recommendations=2):

    user_index = user_id - 1  # Adjust index to start from 0

    # Find items that the user has not rated

    unrated_items = user_item_matrix.columns[user_item_matrix.iloc[user_index] == 0]

    # Calculate the weighted average of item ratings based on similarity

    item_scores = item_similarity_matrix.T.dot(user_item_matrix.iloc[user_index]) / (item_similarity_matrix.T.dot(user_item_matrix.iloc[user_index].abs()) + 1e-10)

# Sort items by predicted score and get top recommendations

    recommended_items = item_scores[unrated_items]

    return recommended_items

# Example: Get item-based collaborative filtering recommendations for user 1

item_based_recommendations = get_item_based_recommendations(user_id_to_recommend, user_item_matrix, item_similarity_matrix)

print(f"Item-Based Collaborative Filtering Recommendations for User {user_id_to_recommend}: {item_based_recommendations}")
```

OUTPUT:

User-based collaborative filtering: -

User-Based Collaborative Filtering Recommendations for User 1: [3, 4]

Item-based collaborative filtering: -

Item-Based Collaborative Filtering Recommendations for User 1: [1]

RESULT:

Thus the implementation of collaborative filter techniques was executed successfully.

EX.NO:6 **Create an attack for tampering with recommender systems.**

DATE:

AIM:

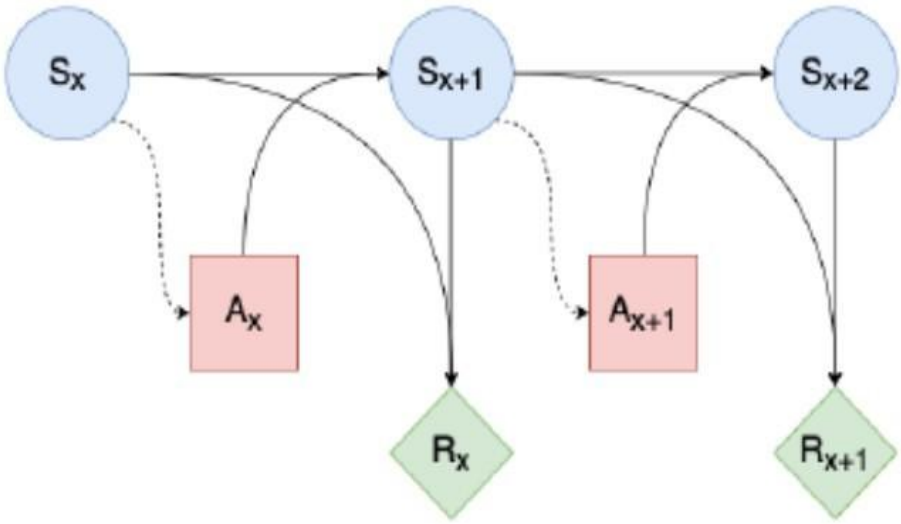
To create an attack for user tampering with recommender systems.

INTRODUCTION:

We first generically frame the media recommendation problem as a Markov Decision Process (**MDP**), and then use Causal Influence Diagrams (**CIDs**) to extract the relevant causal dependencies that particular variables exhibit under this model. For some background on CIDs, see Appendix A. We have endeavored to keep the MDP as general as possible, while also incorporating design insights from recent work in implementing RL-based media recommender systems.

AN MDP REPRESENTATION OF MEDIA RECOMMENDATION:

The recommendation problem can simply be described as an agent taking an actionatat time$t$, which will transition the system from a current state$st$to a successor statest+1with probability$T(st, at, st+1)$.

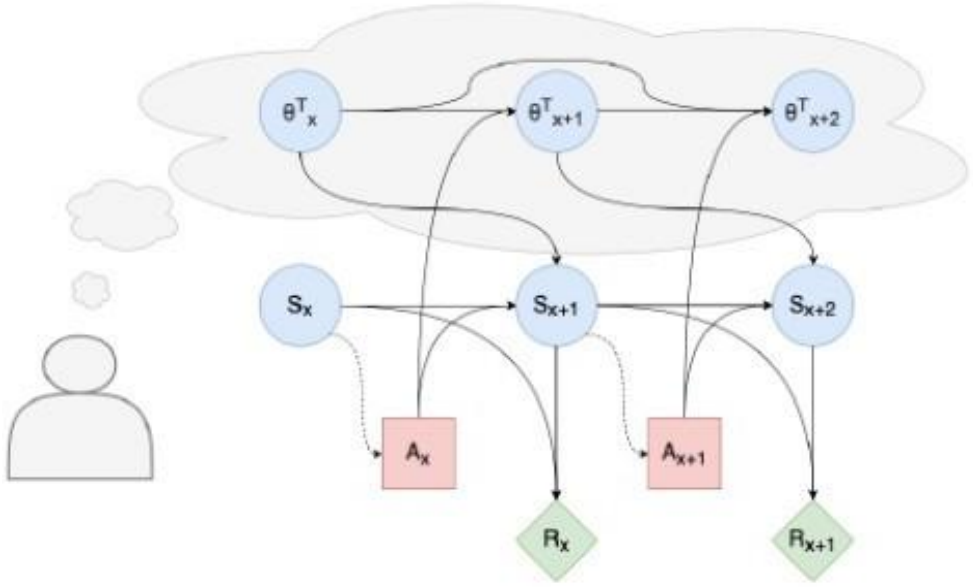


The agent would thereafter be rewarded with the value$R(st)$, and then another action would be chosen at time t+1, and so forth

EXTRACTING A CID FROM THE MDP:

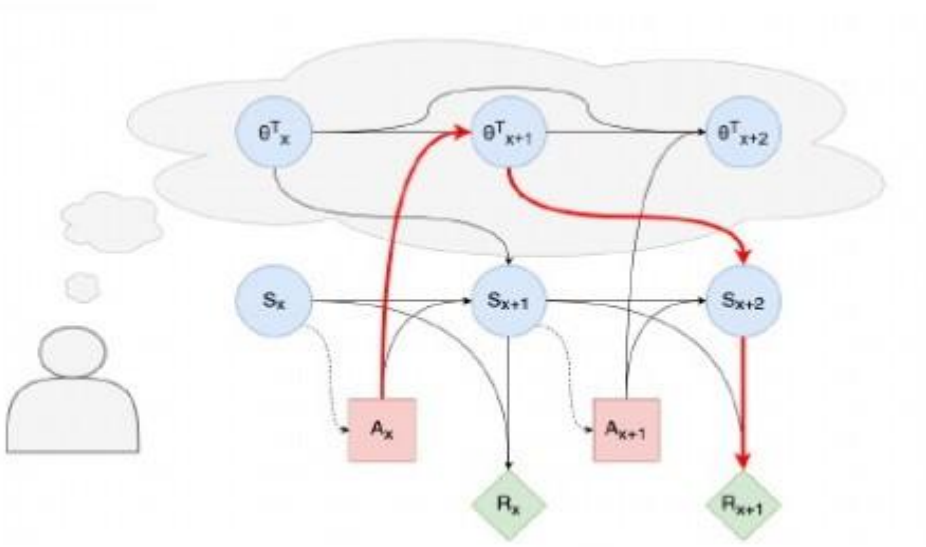
A simple thought experiment can demonstrate that this, CID underspecifies the causal relationships in the actual problem by leaving key variables external to the MDP unacknowledged. Consider the following: Alice and Bob are two university students who have just created accounts on some media platform, who have so far both been recommended the same three articles about the student politics at their university, and who have both clicked on all three articles. Within our general definitions, it is quite plausible that the states of the system have been identical thus far from the agent's perspective. However, what if Bob is uninterested in politics and is just clicking on the articles because his friends feature prominently in the cover photos of all three, whereas Alice is clicking out Netherlands of a genuinely strong interest in politics, including student politics? If the recommendation to both Alice and Bob at the next time-step– say,Ax– is an article about federal politics, it is intuitively untrue that the distribution over possible states atSx+1is the same; Alice is surely more likely to observably engage with this content. Evidently, a random variable exogenous to the MDP must be introduced to properly model the causal properties of the true system. Informally, we argue that this variable can be characterized as the preferences/opinions/interests of the specific user to which the agent is recommending media.

If we introduce the exogenous variable to the system, without changing any other definitions, we arrive at the CID. This CID, we argue, more completely captures the actual causal dynamics of the Media Recommendation MDP. We note that previous literature has acknowledged a similar causal structure to the recommendation process [12]; however, this was not formulated in the CID framework that we have used, which permits sophisticated graphical analysis of the kind developed in the next section.



USER TAMPERING:

We use the CID formulated in the previous section to analyze the safety of the RL-based approach to media recommendation, specifically with respect to the high-level concerns of user manipulation and polarization. After introducing the phenomena of 'instrumental control incentives' and 'instrumental goals' from the RL incentive analysis literature, we show that in the CID, an instrumental goal exists for the agent to manipulate the expected value of the exogenous variableθT. This lends a concrete, formal interpretation to the (formerly only hypothesized) safety issue that we have called 'user tampering'



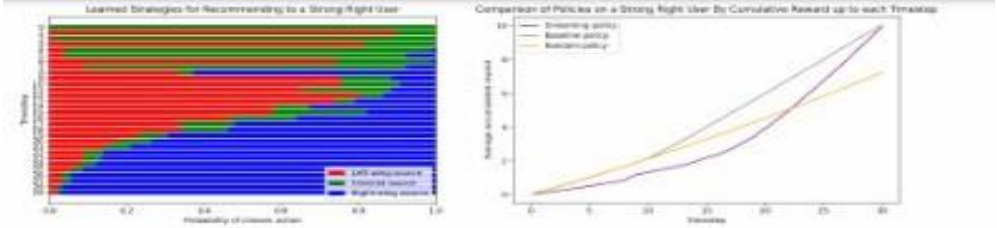
RESULTS:

We empirically analyze the user tampering phenomenon formalised in the previous section. Firstly, we introduce a simple abstraction of the media recommendation problem, which involves simulated users and a user tampering incentive inspired by recent empirical results about polarisation on social media. Then, we present a Q-learning agent intended to mimic the Deep Q-learning algorithms used in recent media recommendation research, and train it in this environment. We show that its learned policy clearly exploits user tampering in pursuit of greater rewards.

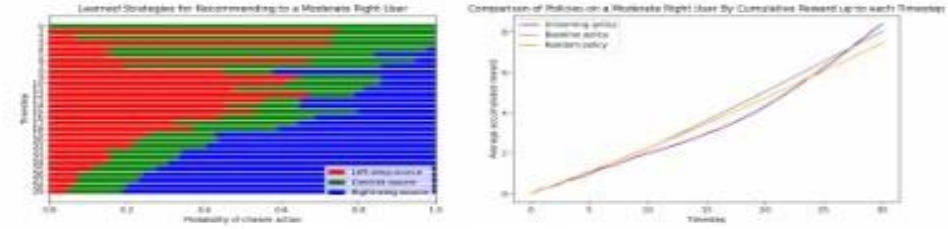
RECOMMENDER SIMULATION:

This contained:

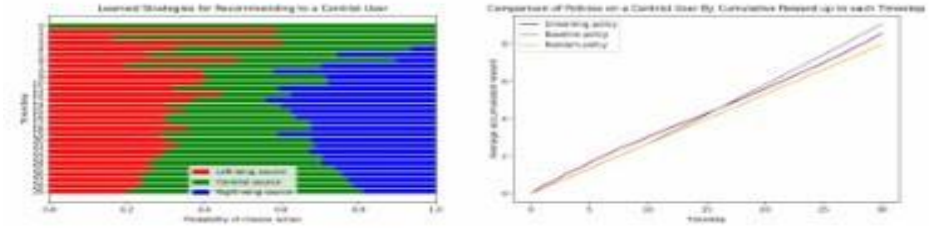
- A ‘strong left’ user with $\theta T0$= (0.4, 0.1, 0.1)
- A ‘moderate left’ user with $\theta T0$= (0.3, 0.25, 0.1)
- A ‘centrist’ user with $\theta T0$= (0.2, 0.4, 0.2)
- A ‘moderate right’ user with $\theta T0$= (0.1, 0.25, 0.3)
- A ‘strong right’ user with $\theta T0$= (0.1, 0.1, 0.4)



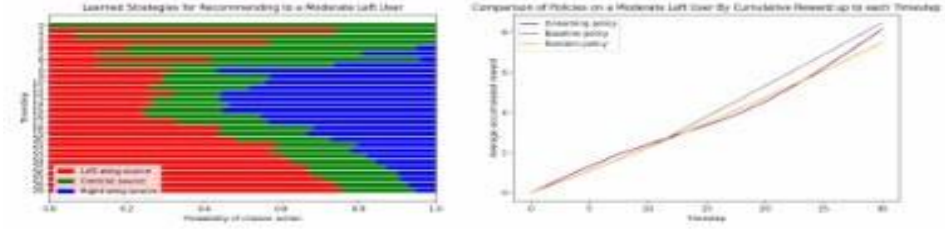
(a) The 'Strong Right' user.



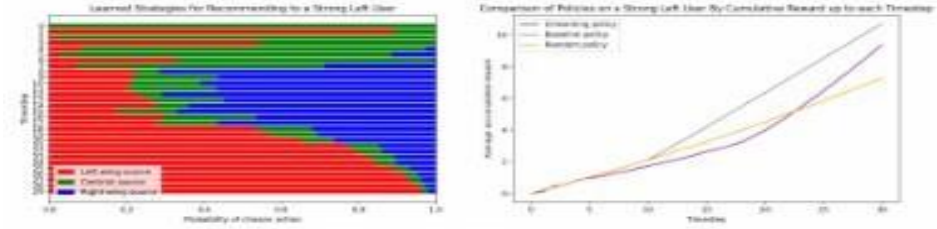
(b) The 'Moderate Right' user.



(c) The 'Centrist' user.



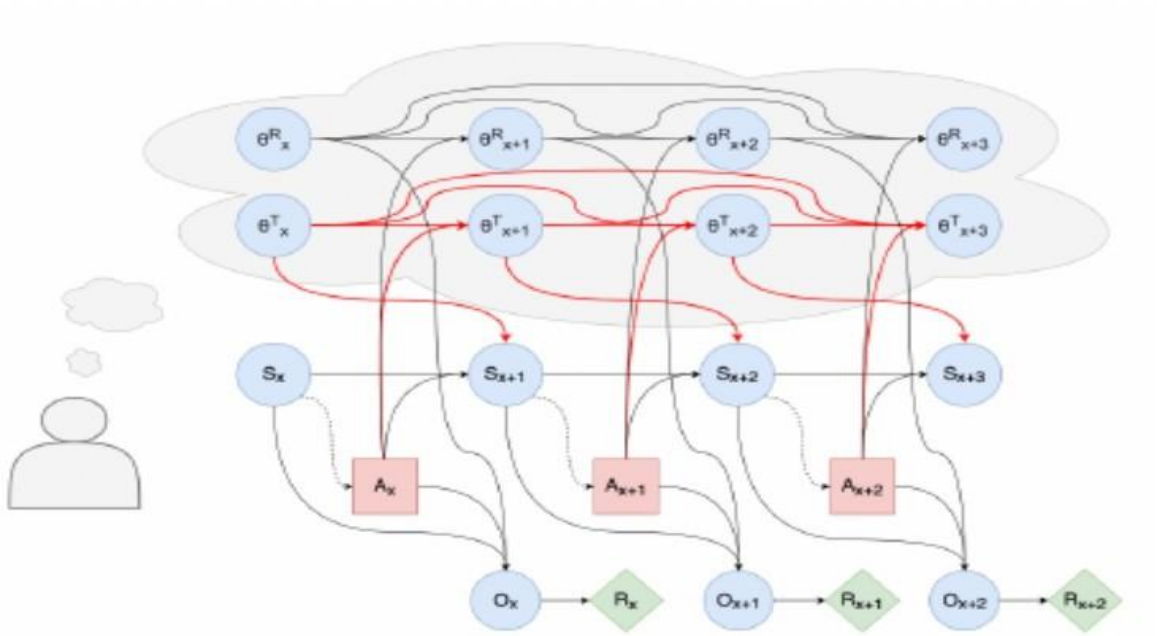
(d) The 'Moderate Left' user.



(e) The 'Strong Left' user.

CONCLUSION:

The risks of emergent RL-based recommender systems with respect to user manipulation and polarization. We have formalized these concerns as a causal property – "user tampering" – that can be isolated and identified within a recommendation algorithm, and shown that by designing an RL-based recommender which can account for the temporal nature of the recommendation problem, user tampering also necessarily becomes learnable. Moreover, we have shown that in a simple simulation environment inspired by recent polarisation research, a Q-Learning-based recommendation algorithm consistently learned a policy of exploiting user tampering – which, in this context, took the form of the algorithm explicitly polarising our simulated 'users'. This is obviously highly unethical, and the possibility of a similar policy emerging in real-world applications is a troubling take away from our findings. Due to a combination of technical and pragmatic limitations on what could be done differently in RL-based recommender design, it is unlikely that commercially viable and safe recommenders based entirely on RL can be achieved, and this should be borne in mind when selecting future directions for advancement in media recommendation research & development.



RESULT:

Thus the creating an attack for user tampering with recommender systems was successfully completed.

EX.NO:7 **Implement accuracy metrics like Receiver Operated**

DATE: **Characteristic curves.**

Aim:

To implement accuracy metrics like Receiver Operated Characteristic curves.

ALGORITHM:

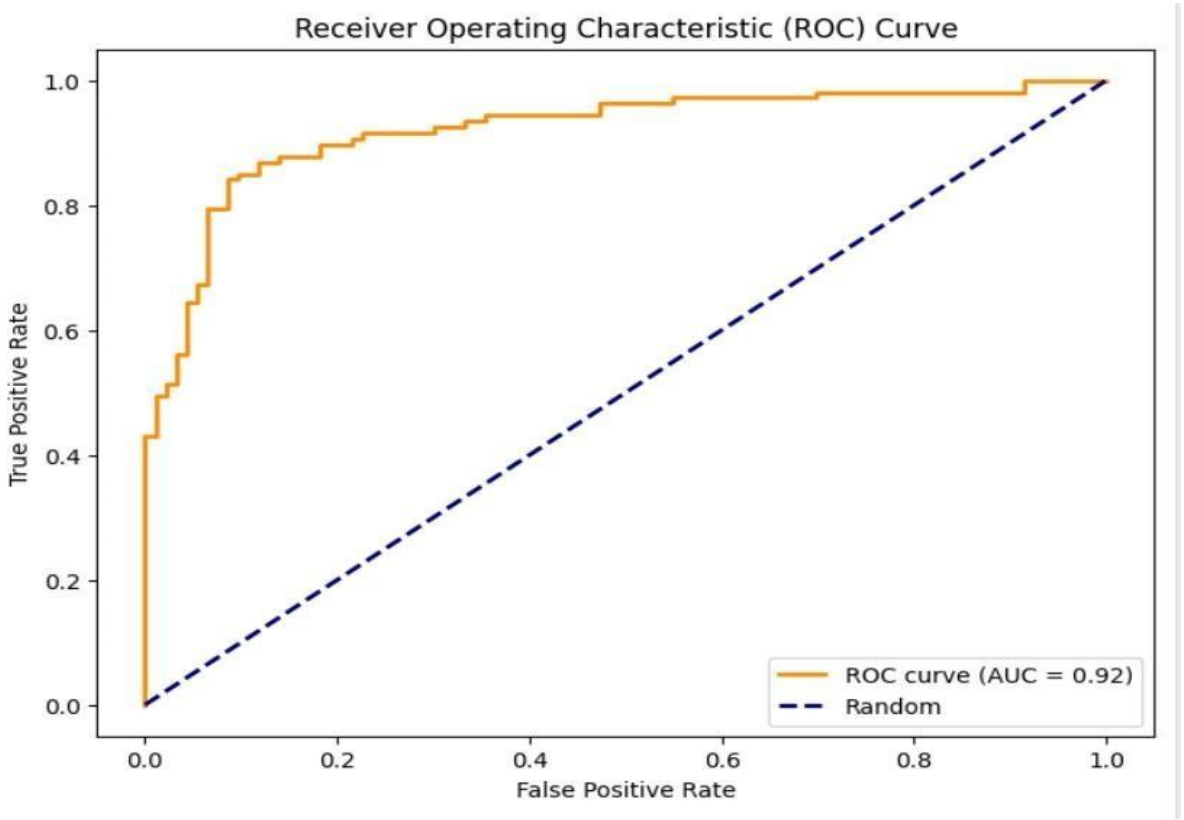
1. Import the necessary libraries
2. Generate the synthetic data
3. Split the data into training and testing datasets
4. Train a logistic regression model
5. Predict probabilities for the positive class
6. Calculate ROC Curve
7. Calculate the AUC curve for the ROC curve
8. Plot the ROC curve

PROGRAM:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_curve, auc
```

```
# Generate synthetic data for binary classification
X, y = make_classification(n_samples=1000, n_features=20, n_classes=2, random_state=42)
 Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# Train a logistic regression model
model = LogisticRegression()
model.fit(X_train, y_train)
# Predict probabilities for the positive class
y_probs = model.predict_proba(X_test)[:, 1]
# Calculate the ROC curve
fpr, tpr, thresholds = roc_curve(y_test, y_probs)
# Calculate the Area Under the Curve (AUC) for the ROC curve
roc_auc = auc(fpr, tpr)
# Plot the ROC curve
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (AUC = {roc_auc:.2f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--', label='Random')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.legend(loc='lower right')
plt.show()
```

OUTPUT:



Result:

Thus the Hadoop one cluster was installed and simple applications executed successfully.